08 - RPC 서비스 및 API 시스템

개요

Geth의 RPC (Remote Procedure Call) 서비스 및 API 시스템은 외부 클라이언트가 이더리움 노드와 상호작용할 수 있는 핵심 인터페이스입니다. 이 시스템은 HTTP, WebSocket, IPC 등 다양한 전송 프로토콜을 지원하며, JSON-RPC 2.0 표준을 따릅니다.

RPC 시스템은 크게 세 가지 주요 컴포넌트로 구성됩니다:

- **RPC 서버** (`rpc.Server`): 요청 처리 및 응답 관리
- **API 엔드포인트** (`ethapi` 패키지): 이더리움 관련 기능 제공
- **클라이언트** (`rpc.Client`): 외부에서 노드에 접근하기 위한 클라이언트

전체 시스템에서 RPC 레이어는 최상위에 위치하여 모든 하위 컴포넌트들의 기능을 외부에 노출시키는 역할을 담당합니다.

핵심 구조체 및 인터페이스

RPC 서버 구조체

```
// Server는 RPC 서버를 나타냅니다
type Server struct {
services serviceRegistry
idgen  func() ID

mutex sync.Mutex
```

```
codecs  map[ServerCodec]struct{}
run atomic.Bool
batchItemLimit  int
batchResponseLimit  int
httpBodyLimit  int
wsReadLimit  int64
}
```

주요 API 구조체들

```
// EthereumAPI는 이더리움 관련 정보에 접근하는 API를 제공합니다
type EthereumAPI struct {
b Backend
}

// TxPoolAPI는 트랜잭션 풀에 대한 API를 제공합니다
type TxPoolAPI struct {
b Backend
}

// BlockChainAPI는 이더리움 블록체인 데이터에 접근하는 API를 제공합니다
type BlockChainAPI struct {
b Backend
```

```
}

// adminAPI는 노드 관리를 위한 관리자 API 모음입니다
type adminAPI struct {
node *Node
}
```

RPC 클라이언트 구조체

```
// Client는 RPC 서버에 대한 연결을 나타냅니다
type Client struct {
idgen  func() ID
isHTTP  bool
services *serviceRegistry

idCounter atomic.Uint32
reconnectFunc reconnectFunc

batchItemLimit  int
batchResponseMaxSize  int

writeConn jsonWriter
close  chan  struct{}
```

```
closing chan struct{}
didClose chan struct{}
// ... 기타 필드들
}
```

주요 프로세스

1. RPC 서버 초기화 및 시작

```
// 새로운 RPC 서버 생성
func NewServer() *Server {
server := &Server{
idgen: randomIDGenerator(),
codecs: make(map[ServerCodec]struct{}),
httpBodyLimit: defaultBodyLimit,
wsReadLimit: wsDefaultReadLimit,
}
server.run.Store(true)
// 기본 RPC 서비스 등록
rpcService := &RPCService{server}
server.RegisterName(MetadataApi, rpcService)
return server
```

```
}
```

2. API 서비스 등록 과정

```
// API 서비스를 특정 네임스페이스에 등록
func (s *Server) RegisterName(name  string, receiver  interface{}) error {
return  s.services.registerName(name, receiver)
}

// 사용 예시
server := rpc.NewServer()
server.RegisterName("eth", &EthereumAPI{backend})
server.RegisterName("admin", &adminAPI{node})
server.RegisterName("txpool", &TxPoolAPI{backend})
```

3. HTTP 요청 처리 흐름

```
// HTTP 요청을 처리하는 메인 핸들러
func (s *Server) ServeHTTP(w http.ResponseWriter, r *http.Request) {
// 빈 GET 요청 허용 (헬스체크용)
if  r.Method == http.MethodGet && r.ContentLength == 0 && r.URL.RawQuery == "" {
w.WriteHeader(http.StatusOK)
return
```

```
}

// 요청 검증

if  code, err := s.validateRequest(r);  err != nil {

http.Error(w, err.Error(), code)

return

}



// 연결 정보 설정

connInfo := PeerInfo{Transport: "http", RemoteAddr: r.RemoteAddr}

connInfo.HTTP.Version = r.Proto

connInfo.HTTP.Host = r.Host

connInfo.HTTP.Origin = r.Header.Get("Origin")

connInfo.HTTP.UserAgent = r.Header.Get("User-Agent")

ctx := context.WithValue(r.Context(), peerInfoContextKey{}, connInfo)



// 코덱 생성 및 단일 요청 처리

w.Header().Set("content-type", contentType)

codec := s.newHTTPServerConn(r, w)

defer  codec.close()

s.serveSingleRequest(ctx, codec)

}
```

4. WebSocket 연결 처리

```go
// WebSocket 핸들러 생성
func (s *Server) WebsocketHandler(allowedOrigins []string) http.Handler {
var  upgrader = websocket.Upgrader{
ReadBufferSize: wsReadBuffer,
WriteBufferSize: wsWriteBuffer,
CheckOrigin: wsHandshakeValidator(allowedOrigins),
}
return  http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
conn, err := upgrader.Upgrade(w, r, nil)
if  err != nil {
log.Debug("WebSocket upgrade failed", "err", err)
return
}
codec := newWebsocketCodec(conn, r.Host, r.Header, s.wsReadLimit)
s.ServeCodec(codec, 0)
})
}
```

코드 예시

1. 기본 RPC 서버 설정

```go
package main
```

```go
import (
"net/http"
"github.com/ethereum/go-ethereum/rpc"
"github.com/ethereum/go-ethereum/internal/ethapi"
)

func  main() {
// RPC 서버 생성
server := rpc.NewServer()
// 배치 제한 설정
server.SetBatchLimits(1000, 25*1024*1024) // 1000개 항목, 25MB 응답 제한
server.SetHTTPBodyLimit(5 * 1024 * 1024) // 5MB HTTP 바디 제한
// API 서비스 등록
ethAPI := &ethapi.EthereumAPI{backend}
server.RegisterName("eth", ethAPI)
adminAPI := &adminAPI{node}
server.RegisterName("admin", adminAPI)
// HTTP 서버 시작
http.Handle("/", server)
http.ListenAndServe(":8545", nil)
}
```

2. 주요 Ethereum API 메서드들

```go
// 가스 가격 조회
func (api *EthereumAPI) GasPrice(ctx context.Context) (*hexutil.Big, error) {
tipcap, err := api.b.SuggestGasTipCap(ctx)
if err != nil {
return nil, err
}
if head := api.b.CurrentHeader(); head.BaseFee != nil {
tipcap.Add(tipcap, head.BaseFee)
}
return (*hexutil.Big)(tipcap), err
}

// 최대 우선순위 수수료 조회
func (api *EthereumAPI) MaxPriorityFeePerGas(ctx context.Context) (*hexutil.Big,
tipcap, err := api.b.SuggestGasTipCap(ctx)
if err != nil {
return nil, err
}
return (*hexutil.Big)(tipcap), err
}
```

```go
// 수수료 히스토리 조회
func (api *EthereumAPI) FeeHistory(ctx context.Context, blockCount math.HexOrDec
lastBlock rpc.BlockNumber, rewardPercentiles []float64) (*feeHistoryResult, erro
// 구현 로직...
}
```

3. 관리자 API 사용 예시

```go
// HTTP RPC 서버 시작
func (api *adminAPI) StartHTTP(host *string, port *int, cors *string,
apis *string, vhosts *string) (bool, error) {
api.node.lock.Lock()
defer  api.node.lock.Unlock()

if  api.node.http != nil {
return false, fmt.Errorf("HTTP RPC already running on %s", api.node.http.listenA
}

// HTTP 서버 구성 및 시작
if  err := api.node.http.setListenAddr(*host, *port);  err != nil {
return false, err
}
if  err := api.node.http.enableRPC(apis, httpConfig{
```

```
CorsAllowedOrigins: cors,
Vhosts: vhosts,
});  err != nil {
return false, err
}
if  err := api.node.http.start();  err != nil {
return false, err
}
return true, nil
}

// WebSocket 서버 시작
func (api *adminAPI) StartWS(host *string, port *int, allowedOrigins *string,
apis *string) (bool, error) {
api.node.lock.Lock()
defer  api.node.lock.Unlock()

// WebSocket 서버 구성 및 시작
if  err := api.node.ws.setListenAddr(*host, *port);  err != nil {
return false, err
}
if  err := api.node.ws.enableWS(rpcEndpointsAll, wsConfig{
Origins: allowedOrigins,
```

```
Modules: apis,

});  err != nil {

return false, err

}

if  err := api.node.ws.start();  err != nil {

return false, err

}

return true, nil

}
```

4. RPC 클라이언트 사용법

```
package main


import (

"context"

"fmt"

"math/big"

"github.com/ethereum/go-ethereum/rpc"

"github.com/ethereum/go-ethereum/common/hexutil"

)


func  main() {
```

```go
// HTTP 클라이언트 연결
client, err := rpc.DialHTTP("http://localhost:8545")
if err != nil {
panic(err)
}
defer client.Close()

// 단일 RPC 호출
var result hexutil.Big
err = client.Call(&result, "eth_gasPrice")
if err != nil {
panic(err)
}
fmt.Printf("Gas Price: %s\n", (*big.Int)(&result).String())

// 배치 RPC 호출
batch := []rpc.BatchElem{
{Method: "eth_gasPrice", Result: &result},
{Method: "eth_blockNumber", Result: new(hexutil.Big)},
}
err = client.BatchCall(batch)
if err != nil {
panic(err)
```

```go
}
for i, elem := range batch {
if elem.Error != nil {
fmt.Printf("Batch call %d failed: %v\n", i, elem.Error)
} else {
fmt.Printf("Batch call %d result: %v\n", i, elem.Result)
}
}
}
```

5. WebSocket 클라이언트 연결

```go
func connectWebSocket() {
// WebSocket 클라이언트 연결
client, err := rpc.DialWebsocket(context.Background(), "ws://localhost:8546", ""
if err != nil {
panic(err)
}
defer client.Close()

// 구독 생성
ch := make(chan map[string]interface{})
sub, err := client.Subscribe(context.Background(), ch, "eth_subscribe", "newHead
```

```go
if err != nil {
panic(err)
}
defer sub.Unsubscribe()

// 새 블록 헤더 수신
for {
select {
case header := <-ch:
fmt.Printf("New block header: %v\n", header)
case err := <-sub.Err():
fmt.Printf("Subscription error: %v\n", err)
return
}
}
}
```

최적화 및 성능

1. 배치 처리 최적화

```go
// 배치 제한 설정으로 성능 최적화
server.SetBatchLimits(1000, 25*1024*1024)
```

```go
// 배치 요청 처리
func (s *Server) serveSingleRequest(ctx context.Context, codec ServerCodec) {
h := newHandler(ctx, codec, s.idgen, &s.services, s.batchItemLimit, s.batchRespo
h.allowSubscribe = false
defer  h.close(io.EOF, nil)

reqs, batch, err := codec.readBatch()
if  err != nil {
return
}
if  batch {
h.handleBatch(reqs) // 배치 처리
} else {
h.handleMsg(reqs[0]) // 단일 요청 처리
}
}
```

2. 연결 관리 최적화

```go
// 코덱 추적을 통한 연결 관리
func (s *Server) trackCodec(codec ServerCodec) bool {
s.mutex.Lock()
```

```go
defer s.mutex.Unlock()


if !s.run.Load() {
return false // 서버가 중단된 경우 서비스 거부
}
s.codecs[codec] = struct{}{}
return true
}


func (s *Server) untrackCodec(codec ServerCodec) {
s.mutex.Lock()
defer s.mutex.Unlock()
delete(s.codecs, codec)
}
```

3. HTTP 타임아웃 설정

```go
// HTTP 타임아웃 구성
var DefaultHTTPTimeouts = HTTPTimeouts{
ReadTimeout: 30 * time.Second,
ReadHeaderTimeout: 30 * time.Second,
WriteTimeout: 30 * time.Second,
IdleTimeout: 120 * time.Second,
```

```
}

// 컨텍스트 기반 타임아웃 처리
func ContextRequestTimeout(ctx context.Context) (time.Duration, bool) {
timeout := time.Duration(math.MaxInt64)
hasTimeout := false
if deadline, ok := ctx.Deadline(); ok {
timeout = time.Until(deadline)
hasTimeout = true
}
// HTTP 서버의 WriteTimeout 고려
if httpSrv, ok := ctx.Value(http.ServerContextKey).(*http.Server); ok && httpS
wt := httpSrv.WriteTimeout - 100*time.Millisecond
if wt < timeout {
timeout = wt
hasTimeout = true
}
}
return timeout, hasTimeout
}
```

디버깅 및 모니터링

1. 로깅 포인트

```go
// RPC 서버 시작/중단 로깅
func (s *Server) Stop() {
s.mutex.Lock()
defer s.mutex.Unlock()

if s.run.CompareAndSwap(true, false) {
log.Debug("RPC server shutting down")
for codec := range s.codecs {
codec.close()
}
}
}

// WebSocket 업그레이드 실패 로깅
func (s *Server) WebsocketHandler(allowedOrigins []string) http.Handler {
return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
conn, err := upgrader.Upgrade(w, r, nil)
if err != nil {
log.Debug("WebSocket upgrade failed", "err", err)
return
}
```

```go
// ...
})
}
```

2. 에러 처리 및 응답

```go
// HTTP 에러 응답 구조체
type HTTPError struct {
Status  string
StatusCode  int
Body []byte
}

func (err HTTPError) Error() string {
return  fmt.Sprintf("%d %s: %s", err.StatusCode, err.Status, err.Body)
}

// 요청 검증 및 에러 응답
func (s *Server) validateRequest(r *http.Request) (int, error) {
if  r.Method == http.MethodPut || r.Method == http.MethodDelete {
return  http.StatusMethodNotAllowed, errors.New("method not allowed")
}
if  r.ContentLength > int64(s.httpBodyLimit) {
```

```
err := fmt.Errorf("content length too large (%d>%d)", r.ContentLength, s.httpBod
return  http.StatusRequestEntityTooLarge, err
}
return  0, nil
}
```

3. 메트릭 수집

```
// 피어 정보 수집
func  PeerInfoFromContext(ctx context.Context) PeerInfo {
info, _ := ctx.Value(peerInfoContextKey{}).(PeerInfo)
return  info
}

// RPC 서비스 모듈 정보
func (s *RPCService) Modules() map[string]string {
s.server.services.mu.Lock()
defer  s.server.services.mu.Unlock()

modules := make(map[string]string)
for  name := range  s.server.services.services {
modules[name] = "1.0"
}
```

```
return modules
}
```

관련 컴포넌트

의존성 관계

1. **하위 의존성**:

- `core` 패키지: 블록체인 데이터 접근
- `eth` 패키지: 이더리움 백엔드 서비스
- `p2p` 패키지: 네트워크 정보
- `accounts` 패키지: 계정 관리

2. **상호작용 방식**:

- Backend 인터페이스를 통한 하위 시스템 접근
- Context를 통한 요청 정보 전달
- 이벤트 구독을 통한 실시간 데이터 스트리밍

보안 고려사항

```
// Origin 검증
func wsHandshakeValidator(allowedOrigins []string) func(*http.Request) bool {
origins := mapset.NewSet[string]()
allowAllOrigins := false

for _, origin := range allowedOrigins {
```

```
if  origin == "*" {
allowAllOrigins = true
}
if  origin != "" {
origins.Add(strings.ToLower(origin))
}
}


return  func(req *http.Request) bool {
if  allowAllOrigins {
return true
}
origin := strings.ToLower(req.Header.Get("Origin"))
return  origins.Contains(origin)
}
}
```

RPC 서비스 및 API 시스템은 Geth 노드의 모든 기능을 외부에 안전하고 효율적으로 노출시키는 핵심 인터페이스로, 다양한 전송 프로토콜과 보안 메커니즘을 통해 안정적인 서비스를 제공합니다.